InkLab

FreeRTOS → Zephyr RTOS

Complete Porting Guide | v1 Hardware | STM32G0B1 → H562 Migration Path

0. Document Overview & How to Use This Guide

This guide provides a complete, step-by-step process to port InkLab's firmware from FreeRTOS (running on the STM32G0B1, v1) to Zephyr RTOS (targeting the STM32H562, v2). It also establishes permanent modularity rules and includes a reusable AI assistant prompt template for all future development sessions.

How to navigate

Phases 1-2: Environment setup and GitHub strategy. Do these before writing any firmware.
Phase 3: Module porting order with explicit peripheral-by-peripheral steps.
Phase 4: Protocol migration (JSON → COBS binary).
Phase 5: Modularity rules and documentation standards.
Phase 6: The master AI session prompt — copy-paste into any new conversation.

1. Tools & Prerequisites

1.1 Host Machine Requirements

Tool	Purpose / Install Command
Python 3.10+	Required by west and Zephyr scripts
CMake 3.20+	Build system — brew install cmake / apt install cmake
Ninja	Fast build backend — brew install ninja / apt install ninja-build
west (Zephyr meta-tool)	pip3 install west
Zephyr ARM toolchain	Download from docs.zephyrproject.org/latest/develop/toolchains/
git 2.30+	Standard version control
VS Code	Recommended IDE
nRF Connect for VS Code	VS Code extension — provides build/flash/debug GUI
J-Link or STLink probe	For v1 flashing; BMP FPGA core replaces this on v2
OpenOCD 0.12+	Debug server — only needed if not using nRF Connect

1.2 VS Code Extensions

- nRF Connect for VS Code (Nordic Semiconductor) — primary build/debug extension
- C/C++ IntelliSense (Microsoft)
- Cortex-Debug — for raw GDB sessions if needed
- GitLens — enhances branch and blame workflows
- Error Lens — inline compiler errors

> **Important: Abandon STM32CubeIDE for all Zephyr work**
> CubeIDE and CubeMX are incompatible with the Zephyr build model. All pin configuration, peripheral setup,
> and clock trees are described in Device Tree (.dts) files — not in GUI tools.
> Keep CubeMX only as a reference schematic viewer. Never generate code from it for the Zephyr branch.

2. GitHub Repository Strategy

2.1 Branch Structure

The FreeRTOS codebase remains on main as the stable v2 release. All Zephyr work happens in a parallel branch. This allows v2 hardware bringup (power path, EPD, BMP validation) to continue on the FreeRTOS branch without blocking the port.

```
# One-time setup — from inside the existing repo
git checkout main
git pull
git checkout -b zephyr/port-base

# Each ported module gets its own short-lived branch
git checkout -b zephyr/port-power-monitor    # example
# ... port, test, commit ...
git checkout zephyr/port-base
git merge --no-ff zephyr/port-power-monitor
git branch -d zephyr/port-power-monitor
```

2.2 Recommended Branch Naming

Branch	Purpose
main	FreeRTOS stable — always buildable — v2 hardware development continues here
zephyr/port-base	Integration branch for all Zephyr work — periodically synced from main
zephyr/port-<module>	Short-lived branch per ported module, merged via --no-ff PR
zephyr/wamr-integration	Long-running branch for WAMR task (risky, isolated)
zephyr/binary-protocol	COBS telemetry migration (also risky, isolated)

2.3 Commit Message Convention

Adopt a consistent prefix convention immediately. This makes git log readable as a changelog.

```
port(power_monitor): replace HAL_I2C with Zephyr i2c_burst_read_dt
port(fpga): add SPI device node to inklab_v2.dts
port(usb-cdc): migrate CDC_Transmit_FS to uart_fifo_fill
docs(api): add @brief/@param/@return to all public functions in bq25798.h
fix(telemetry): correct EMA coefficient after binary protocol switch
chore(west): update Zephyr SDK to 0.17.0
feat(wamr): initial WAMR task scaffold on Zephyr
```

2.4 GitHub Actions CI (Recommended)

Add a minimal CI workflow early. It catches broken builds before they reach main.

```yaml
# .github/workflows/zephyr-build.yml
name: Zephyr Build
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    container: ghcr.io/zephyrproject-rtos/ci:latest
    steps:
      - uses: actions/checkout@v4
      - run: west init -l .
      - run: west update
      - run: west build -b nucleo_h562zi app -- -DCONF_FILE=prj.conf
```

2.5 Workspace Directory Structure

Zephyr uses an "application out of tree" model. Your firmware repo is a dependency consumer of Zephyr, not the Zephyr repo itself.

```
inklab-workspace/            # Created by west init — do NOT git init this
  inklab-firmware/           # Your repo (cloned here by west)
    app/                     # Application source
      src/                   # All .c files (ported modules)
      include/               # All .h files (public APIs)
      boards/                # Board-specific DTS overlays
        inklab_v2.dts        # V2 pin assignments, I2C, SPI, USB, SD
        inklab_v1.dts        # V1 board (for migration testing)
      drivers/               # Out-of-tree driver stubs (if needed)
      CMakeLists.txt
      prj.conf               # Kconfig: enable FreeRTOS/Zephyr modules
      west.yml               # Declares Zephyr version dependency
  zephyr/                    # Zephyr kernel (managed by west update)
  modules/                   # HAL shims, FATFS, etc (managed by west)
  .west/                     # west metadata
```

> **Rule: Never commit generated files**
> The zephyr/, modules/, and .west/ directories are fully managed by west.

Add them to .gitignore. Only your app/ directory belongs in version control.

3. Module Porting Order — Step by Step

The order below is deliberate: each step exercises the minimum new surface area, produces a clear pass/fail result, and unlocks the next step. Never skip ahead — a subtle clock or DMA misconfiguration caught at step 3 will silently corrupt everything built on top of it.

Phase A — Toolchain & Board Bring-Up (No module code yet)

A.1 — West workspace init & blinky on nucleo_h562zi

Goal: confirm the Zephyr toolchain can build and flash to any STM32H562 target. Use the Nucleo eval board if v2 hardware has not arrived.

```
mkdir inklab-workspace && cd inklab-workspace
west init -m https://github.com/your-org/inklab-firmware --mr zephyr/port-base
west update
west build -b nucleo_h562zi zephyr/samples/basic/blinky
west flash
```

- Pass criterion: LED blinks. If it does not, fix toolchain before writing any module code.

A.2 — Author inklab_v1.dts (test board overlay)

Write the Device Tree overlay for the v1 board (STM32G0B1) while the v2 schematic is being reviewed. This gives a real target to validate I2C, SPI, and USB nodes against. The structure you define here transfers directly to inklab_v2.dts.

```
// boards/inklab_v1.dts  (overlay on top of stm32g0b1ceut6)
/ {
    chosen {
        zephyr,console = &usart1;
        zephyr,shell-uart = &usart1;
    };
};

&i2c1 {
    status = "okay";
    clock-frequency = <I2C_BITRATE_FAST>;

    power_monitor_0: ina219@40 {
        compatible = "ti,ina219";
        reg = <0x40>;
        shunt-resistor = <100000>; /* 100 mOhm in uOhm */
    };
};
```

```
&spi1 {
    status = "okay";
    cs-gpios = <&gpioa 4 GPIO_ACTIVE_LOW>;
    fpga_spi: fpga@0 { compatible = "generic-spi-device"; reg = <0>; ... };
};
```

- Pass criterion: west build compiles without DTS errors.

A.3 — GPIO output test (LED blink on actual v1 board)

Before touching any peripheral library, verify that gpio_pin_set_dt() works on your exact hardware. This confirms the DTS pin assignment is correct and the GPIO clock is enabled.

```
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

static const struct gpio_dt_spec led =
    GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);

int main(void) {
    gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
    while (1) {
        gpio_pin_toggle_dt(&led);
        k_sleep(K_MSEC(500));
    }
}
```

- Pass criterion: LED toggles at 1 Hz. Oscilloscope confirms timing.

Phase B — Core Peripherals

B.1 — I2C: port powerMonitor.c (INA219)

This is the reference module for the entire port. It is pure I2C + RTOS mutex — no DMA, no interrupts, no complex state. Port it first to establish the HAL-to-Zephyr translation pattern that all subsequent drivers will follow.

STM32 HAL (before)	Zephyr API (after)
HAL_I2C_Mem_Read(&hi2c1, addr, reg, 1, buf, len, HAL_MAX_DELAY)	i2c_burst_read_dt(&i2c_dev, reg, buf, len)
HAL_I2C_Mem_Write(&hi2c1, addr, reg, 1, buf, len, HAL_MAX_DELAY)	i2c_burst_write_dt(&i2c_dev, reg, buf, len)
osMutexAcquire(i2c1_mutex, osWaitForever)	k_mutex_lock(&i2c1_mutex, K_FOREVER)
osMutexRelease(i2c1_mutex)	k_mutex_unlock(&i2c1_mutex)
osDelay(10)	k_sleep(K_MSEC(10))

The PowerMonitor_t handle struct stores a const struct device * (from DEVICE_DT_GET) instead of I2C_HandleTypeDef *. Initialize it in the module init function, not in main.

- Pass criterion: Power readings appear over RTT/UART within 2% of known good values.

B.2 — I2C: port bq25798.c

Same pattern as powerMonitor.c but more registers. Port the I2C read/write calls first. Do not touch BQ25798_GetJson() yet — that gets replaced in Phase D.

- Pass criterion: BQ25798_ReadAll() returns valid voltage/current/status over USB shell.

B.3 — SPI + DMA: port fpga.c

The FPGA communicates over SPI with DMA. This is the most performance-sensitive peripheral on the platform. Get the SPI working without DMA first, then add DMA.

```
/* Step 1 — polled SPI, no DMA */
struct spi_buf tx_buf = { .buf = pkt, .len = len };
struct spi_buf_set tx_set = { .buffers = &tx_buf, .count = 1 };
spi_transceive_dt(&fpga_spi, &tx_set, NULL);

/* Step 2 — async SPI with DMA */
/* Enable CONFIG_SPI_ASYNC=y in prj.conf */
spi_transceive_signal_dt(&fpga_spi, &tx_set, NULL, &done_signal);
k_poll(&done_event, 1, K_FOREVER);
```

Performance note: SPI DMA path

The FreeRTOS version uses HAL_SPI_Transmit_DMA with a binary semaphore in the DMA complete callback.

In Zephyr, use spi_transceive_signal_dt + k_poll on a k_poll_signal. This is equivalent throughput

with zero HAL overhead. Confirm DMA channel assignment does not conflict with SD card SPI.

- Pass criterion: FPGA_Program_Slot(0) loads bitstream correctly. Logic analyzer confirms SPI framing.

B.4 — GPIO inputs: Joystick

The 5-way joystick is polled via ADC or GPIO depending on v1 wiring. Port the read function and confirm DTS alias resolves.

```
/* If analog: use Zephyr ADC subsystem */
#include <zephyr/drivers/adc.h>
static const struct adc_dt_spec joy_adc =
    ADC_DT_SPEC_GET_BY_NAME(DT_PATH(joystick), ud);

/* If GPIO matrix: */
static const struct gpio_dt_spec joy_up =
```

```
    GPIO_DT_SPEC_GET(DT_NODELABEL(joy_up), gpios);
```

- Pass criterion: All 5 directions register in the DevConsole Device IO tab.

B.5 — PWM: LED Manager

The RGB LED uses PWM. Replace TIM HAL calls with the Zephyr PWM API.

```
#include <zephyr/drivers/pwm.h>
static const struct pwm_dt_spec led_r =
    PWM_DT_SPEC_GET(DT_ALIAS(pwm_led_r));

pwm_set_dt(&led_r, PWM_MSEC(1), PWM_USEC(brightness_r));
```

- Pass criterion: LED Manager patterns (idle, busy, error) are visually correct.

Phase C — RTOS Task Architecture

C.1 — Thread definitions and priorities

In Zephyr, threads are defined with K_THREAD_DEFINE() at link time or k_thread_create() at runtime. Replace all osThreadNew() calls. Keep the same priority hierarchy — do not optimize priorities until the full system is running.

FreeRTOS (before)	Zephyr (after)
osThreadNew(USB_Task, NULL, &usb_attr)	K_THREAD_DEFINE(usb_thread, 2048, usb_task_fn, NULL, NULL, NULL, 5, 0, 0)
osThreadNew(FPGA_Task, NULL, &fpga_attr)	K_THREAD_DEFINE(fpga_thread, 2048, fpga_task_fn, NULL, NULL, NULL, 4, 0, 0)
osThreadNew(Telemetry_Task, ...)	K_THREAD_DEFINE(telem_thread, 1024, telem_task_fn, NULL, NULL, NULL, 6, 0, 0)
osThreadNew(Joystick_Task, ...)	K_THREAD_DEFINE(joy_thread, 512, joy_task_fn, NULL, NULL, NULL, 7, 0, 0)
osThreadNew(LED_Task, ...)	K_THREAD_DEFINE(led_thread, 512, led_task_fn, NULL, NULL, NULL, 8, 0, 0)

C.2 — Message queues and mutexes

```
/* FreeRTOS */
osMessageQueueId_t telem_q = osMessageQueueNew(8, sizeof(TelemMsg_t), NULL);
osMessageQueuePut(telem_q, &msg, 0, osWaitForever);
osMessageQueueGet(telem_q, &msg, NULL, osWaitForever);

/* Zephyr equivalent */
K_MSGQ_DEFINE(telem_q, sizeof(TelemMsg_t), 8, 4);
```

```
k_msgq_put(&telem_q, &msg, K_FOREVER);
k_msgq_get(&telem_q, &msg, K_FOREVER);

/* Mutex */
K_MUTEX_DEFINE(i2c1_mutex);
k_mutex_lock(&i2c1_mutex, K_FOREVER);
k_mutex_unlock(&i2c1_mutex);
```

C.3 — USB CDC: port USB Task

This is the most impactful single change for the researcher-facing interface. The FreeRTOS version calls CDC_Transmit_FS() from usbd_cdc_if.c (CubeMX generated). In Zephyr, USB CDC ACM is a UART device node.

```
/* prj.conf */
CONFIG_USB_DEVICE_STACK=y
CONFIG_USB_CDC_ACM=y
CONFIG_UART_INTERRUPT_DRIVEN=y

/* In source */
#include <zephyr/usb/usb_device.h>
#include <zephyr/drivers/uart.h>

static const struct device *cdc_dev =
    DEVICE_DT_GET(DT_NODELABEL(cdc_acm_uart0));

/* Transmit (replaces CDC_Transmit_FS) */
uart_fifo_fill(cdc_dev, (uint8_t*)buf, len);

/* Receive callback (replaces USB_ReceiveCallback in usbd_cdc_if.c) */
uart_irq_callback_user_data_set(cdc_dev, cdc_rx_cb, NULL);
uart_irq_rx_enable(cdc_dev);
```

8KB RX ring buffer

The existing 8KB static ring buffer in app_freertos.c can be kept exactly as-is.
Only the fill function changes from CDC_Receive_FS callback to uart_irq_callback.
The ring buffer and command parser (frontend_api.c) are zero-change.

- Pass criterion: Type any command in DevConsole Terminal. Device responds. No character drops under continuous telemetry stream.

C.4 — SD card / FATFS

Zephyr wraps ELM FatFs under its own disk/FS subsystem. The f_open / f_read / f_write calls at the application level are identical — only the mount and init sequence changes.

```
/* prj.conf */
CONFIG_FAT_FILESYSTEM_ELM=y
CONFIG_DISK_DRIVER_SDMMC=y
CONFIG_FILE_SYSTEM=y
```

```
/* Replaces app_fatfs.c MX_FATFS_Init() */
#include <zephyr/fs/fs.h>

static struct fs_mount_t sd_mount = {
    .type = FS_FATFS,
    .fs_data = NULL,
    .mnt_point = "/SD:",
};
fs_mount(&sd_mount);

/* All f_open / f_read / f_write calls below this point: UNCHANGED */
```

The custom user_diskio_spi.c (low-level SPI SD driver) can be reused as a Zephyr disk driver shim or replaced by CONFIG_DISK_DRIVER_SDMMC if the SD is wired to an SDMMC peripheral. On v1 (SPI-wired SD), keep user_diskio_spi.c and register it as a Zephyr disk backend.

- Pass criterion: FPGA slot scan finds all bitstream files on first boot. Write + read speed test passes.

Phase D — Validation & Performance Gates

D.1 — Heap & Stack audit (critical before WAMR)

Run this before committing to WAMR integration. Zephyr provides CONFIG_THREAD_ANALYZER to dump per-thread stack usage.

```
/* prj.conf */
CONFIG_THREAD_ANALYZER=y
CONFIG_THREAD_ANALYZER_USE_PRINTK=y
CONFIG_THREAD_ANALYZER_AUTO=y
CONFIG_THREAD_ANALYZER_AUTO_INTERVAL=10

/* Output (example) */
  usb_thread      : unused 412 / total 2048  (20% used)
  fpga_thread     : unused 680 / total 2048  (33% used)
  telem_thread    : unused 512 / total 1024  (50% used)
```

Target: each thread should have at least 30% stack headroom at peak load. Reduce oversized stacks before adding WAMR.

- Gate: Total committed RAM (threads + queues + static) must be under 490 KB, leaving 150 KB for WAMR interpreter.

D.2 — Telemetry throughput test

Verify the USB CDC ACM path can sustain the same throughput as the FreeRTOS version. Use the existing Power Profiler tab as the benchmark tool.

- Pass criterion: Power Profiler streams 4 rails at 10 Hz with no missed frames. Measured with Wireshark USB capture.

D.3 — FPGA bitstream slot switching under load

Slot switching must be tested while the telemetry task is active. This validates the SPI mutex protects against race conditions between the FPGA task and the telemetry task.

- Pass criterion: 100 back-to-back slot switches without a single corrupt bitstream. No FPGA nDONE failures.

D.4 — BMP/SWD core validation

Load the SWD RTL bitstream and verify the FPGA can program a known-good SWD target through the BMP wrapper Zephyr task.

- Pass criterion: 64 KB firmware upload completes without error at >= 100 KB/s throughput.

4. Protocol Migration: JSON → COBS-Framed Binary

4.1 Why JSON Is the Wrong Choice for This System

The current protocol sends newline-delimited JSON for all telemetry. Looking at the Vue frontend (systemStore.js), the processIncomingData() function handles two distinct stream types that are already incompatible:

- Structured telemetry — { "type": "ina", ... } parsed with JSON.parse()
- Unstructured plaintext — slot names, bench results, FW info, SD log lines parsed with startsWith() and regex

This mixed approach has real costs: text framing is 3-5x larger than binary for float data, JSON.parse on every telemetry frame adds browser GC pressure at high rates, partial frame corruption silently drops data (try/catch eats errors), and text regexes are fragile maintenance burdens.

4.2 Recommended Replacement: COBS-Framed Binary Packets

COBS (Consistent Overhead Byte Stuffing) is the correct protocol for this use case. It is used in production embedded telemetry systems (ArduPilot MAVLink, Zephyr MCUMGR, OpenLog Artemis) for exactly these reasons:

Property	COBS Binary vs JSON Text
Frame delimiter	0x00 byte — unambiguous, no escaping needed
Overhead	1 byte per 254 bytes of payload (< 0.4%)
Float encoding	4 bytes exact — JSON uses 8-14 bytes per float

Property	COBS Binary vs JSON Text
Parse complexity	memcpy into struct — zero allocation, zero GC
Corruption detection	CRC16 field per packet — detectable, not silent
Partial frame handling	Discard on 0x00 boundary — clean recovery
Mixed stream problem	Eliminated — all messages are typed binary packets

4.3 Packet Structure

```
/* inklab_proto.h — shared between firmware and a TS/JS code-gen script */

#pragma pack(push, 1)
typedef struct {
    uint8_t  msg_type;    /* See InkLabMsgType_t enum */
    uint16_t seq;         /* Wrapping sequence number — detect drops */
    uint16_t payload_len; /* Bytes in payload[] */
    uint8_t  payload[];   /* Variable-length, type-specific struct */
    /* uint16_t crc16;    appended after payload by framer */
} InkLabPacket_t;
#pragma pack(pop)

typedef enum {
    MSG_INA_TELEM      = 0x01,
    MSG_BQ_TELEM       = 0x02,
    MSG_SYS_STATUS     = 0x03,
    MSG_GPIO_STATE     = 0x04,
    MSG_FPGA_SLOT      = 0x05,
    MSG_LOG_STRING     = 0x10, /* UTF-8 string — replaces plain text lines */
    MSG_CMD_RESPONSE   = 0x11, /* ACK/NAK with optional payload */
    MSG_BENCH_RESULT   = 0x12,
} InkLabMsgType_t;
```

4.4 Migration Strategy — No Big Bang

Do not rewrite everything at once. The frontend already dispatches on data.type for JSON and on startsWith() for plaintext. Add a mode flag:

1. Add CONFIG_PROTO_BINARY=n to prj.conf. Default off — JSON still works.
2. Implement the COBS framer as a standalone module (cobs.c / cobs.h). Zero dependencies.
3. Add a USB command PROTO:SET:BINARY to enable binary mode at runtime.
4. In the Vue frontend, detect binary frames by checking if the first byte of a received chunk is 0x00 or a known type byte (not ASCII {). Route to a binary parser.
5. Port telemetry types one by one: INA first (most frequent), then BQ, then SYS_STATUS.
6. Keep MSG_LOG_STRING for all existing plaintext lines — this eliminates the startsWith() fragmentation.

7. Once all types are binary, set CONFIG_PROTO_BINARY=y as default and remove JSON formatters.

4.5 Frontend Integration

The processIncomingData() function in systemStore.js currently does: if line starts with { → JSON.parse. Replace with:

```
// In useSerial.js — raw byte handler
function processRawBytes(chunk: Uint8Array) {
  cobsDecoder.push(chunk);  // cobsDecoder buffers until 0x00 delimiter
}

// cobsDecoder emits complete decoded packets:
cobsDecoder.on("packet", (buf: Uint8Array) => {
  const msgType = buf[0];
  const seq     = (buf[2] << 8) | buf[1];   // little-endian
  const paylen  = (buf[4] << 8) | buf[3];
  const payload = buf.slice(5, 5 + paylen);
  const crc     = (buf[5 + paylen + 1] << 8) | buf[5 + paylen];

  if (!verifyCrc16(buf.slice(0, 5 + paylen), crc)) return; // drop corrupt

  binaryHandlers[msgType]?.(payload, store);
});
```

Backward compatibility

During migration, the firmware can emit both formats: JSON when PROTO:SET:JSON is active,
binary when PROTO:SET:BINARY. The frontend auto-detects based on the first byte of each WebSerial
chunk (0x7B = ASCII { means JSON, anything else means binary). No breaking change.

5. Modularity Rules & Documentation Standards

5.1 The Three Laws of InkLab Modules

Law 1 — No HAL in the header

Public .h files must not include stm32g0xx_hal.h, stm32h5xx_hal.h, or any RTOS header. Headers expose types and function signatures only. HAL types (I2C_HandleTypeDef, etc.) are
replaced by opaque const struct device * in the Zephyr port. This makes every module compilable in a host-side unit test without a cross-compiler.

Law 2 — One module, one responsibility
A module init function initializes exactly one hardware resource or logical subsystem. powerMonitor.c initializes INA sensors. bq25798.c initializes the charger. Neither touches the other's peripheral. Shared I2C bus access is mediated only by a mutex defined in a dedicated bus_manager.c — never by one module calling into another's internals.

Law 3 — Config, not magic numbers
Every constant that a researcher might want to change (telemetry rate, PGA gain, bitstream slot
count, WAMR heap size) is a #define or Kconfig symbol in a single config header.
No raw numbers in .c files. No "I'll just hardcode 16 here" comments.

5.2 Directory Layout (Enforced)

```
app/
  src/
    hal/          # Hardware abstraction — one file per IC (bq25798.c, ina4235.c)
    drivers/      # Peripheral drivers — fpga.c, sd_card.c, epd.c
    rtos/         # Task definitions — usb_task.c, fpga_task.c, telem_task.c
    proto/        # Protocol layer — cobs.c, inklab_proto.c, frontend_api.c
    app/          # Application logic — battery_soc.c, led_manager.c, sys_power.c
    wasm/         # WAMR host — wasm_host.c, native_bindings.c
  include/
    hal/          # Public HAL headers
    drivers/
    rtos/
    proto/
    app/
    wasm/
  boards/
    inklab_v1.dts
    inklab_v2.dts
  prj.conf
  CMakeLists.txt
```

5.3 Mandatory Documentation Standard

Every public function (declared in a .h file) must have a Doxygen comment block. No exceptions. Private (static) functions in .c files should have at minimum a one-line description.

```
/**
 * @brief  Initialize the INA power monitor for a specific rail.
 *
 * Configures the INA device, calibrates the current LSB based on the
 * provided shunt resistor value, and verifies the device is reachable
 * on the I2C bus. Must be called before PowerMonitor_Read().
```

```
 *
 * @param  dev         Pointer to the PowerMonitor handle to initialize.
 * @param  i2c_dev     Zephyr I2C device pointer (from DEVICE_DT_GET).
 * @param  addr        7-bit I2C address of the INA device.
 * @param  r_shunt_ohms  Physical shunt resistor value in Ohms.
 * @param  max_expected_A  Maximum expected current in Amperes (sets PGA range).
 *
 * @retval 0           Success.
 * @retval -ENODEV     Device not found on I2C bus.
 * @retval -EIO        I2C communication error during calibration write.
 *
 * @note   This function is NOT thread-safe. Call from a single init context
 *         before RTOS threads start, or protect with a mutex.
 */
int PowerMonitor_Init(PowerMonitor_t *dev, const struct device *i2c_dev,
                      uint8_t addr, float r_shunt_ohms, float max_expected_A);
```

5.4 Return Value Convention

All functions that can fail must return int (0 = success, negative errno = failure). Never return HAL_StatusTypeDef in the Zephyr port. Use standard errno values: -EIO for bus errors, -EINVAL for bad parameters, -ENOMEM for allocation failures. This makes error codes portable and compatible with Zephyr logging macros.

5.5 Zephyr-Specific Rules

- Never call k_sleep() or k_msleep() from an interrupt context. Use k_work or k_timer instead.
- Never use __disable_irq() / __enable_irq() directly. Use Zephyr IRQ lock API: irq_lock() / irq_unlock().
- Mark all thread stack arrays with K_THREAD_STACK_DEFINE, not plain arrays. This ensures correct alignment on Cortex-M33 with TrustZone.
- All I2C and SPI device handles must be acquired at module init time with DEVICE_DT_GET, not inside task loops. Fail fast at boot, not at runtime.
- GPIO toggle safety floor remains 5 ms (100 Hz max). Document this explicitly in frontend_api.h with a @note Doxygen comment on the GPIO command handler.

6. Master AI Session Prompt Template

Copy the following block into any new AI conversation before asking a firmware question. Fill in the [CURRENT TASK] section. The more specific you are about what you are attempting right now, the more precise the AI guidance will be.

```
=== INKLAB PROJECT CONTEXT ===
```

```
PROJECT: InkLab — open-source heterogeneous embedded research platform.
HARDWARE:
  - V1 (active dev): STM32G0B1CEUT6, Cortex-M0+, 64 MHz, 256 KB Flash, 144 KB RAM
  - V2 (arriving): STM32H562RIV, Cortex-M33, 250 MHz, 2 MB Flash (dual-bank), 640
KB RAM
  - FPGA: Lattice iCE40UP5K — SPI3 with DMA — OSS CAD Suite
(Yosys/nextpnr/IceStorm)
  - Display: E-paper (EPD) via parallel 8-bit from FPGA — LVGL/1bpp MCU rendering
  - Power: BQ25798 (USB-C 5V, UPS path) + INA4235 + 4x INA series rails
  - Storage: SD card via SPI + FATFS — 16 named FPGA bitstream slots

RTOS STATUS:
  - Currently running: FreeRTOS (CMSIS-RTOS2 API)
  - Migration target: Zephyr RTOS (in parallel branch zephyr/port-base)
  - Active tasks: USB, FPGA, Telemetry, Joystick, LED
  - Planned tasks: EPD, Wasm (WAMR)

USB INTERFACE:
  - Full-speed CDC — currently JSON over newline framing
  - Migrating to COBS-framed binary (inklab_proto.h)
  - 8 KB RX ring buffer — command parser in frontend_api.c

WEB FRONTEND:
  - Vue.js DevConsole — WebSerial API — Config / Debug / App modes
  - Protocol parser in stores/systemStore.js (processIncomingData)
  - DO NOT suggest rewriting the frontend — it is not in scope

MODULARITY RULES (ENFORCE THESE):
  - No HAL headers in public .h files
  - All public functions must have @brief @param @retval Doxygen comments
  - Return int with errno values (not HAL_StatusTypeDef)
  - Config values in #define/Kconfig, never raw numbers in .c
  - One module = one responsibility

KEY SOURCE FILES:
  app_freertos.c    — task init (being ported to Zephyr K_THREAD_DEFINE)
  frontend_api.c    — USB command parser (zero-change in port)
  powerMonitor.c    — INA219 driver (REFERENCE PORT — do this first)
  bq25798.c         — charger driver
  fpga.c            — SPI/DMA FPGA interface
  pinmux.c          — being replaced by inklab_v2.dts device tree
  battery_soc.c, sys_power.c, led_manager.c, joystick.c — app layer

KNOWN RISKS TO KEEP IN MIND:
  1. RAM is tight: WAMR needs ~150 KB — audit before adding it
  2. .noinit section: verify it is outside BSS in the Zephyr linker script
  3. BQ25798 25ms backup recovery — test USB→battery→USB transition under load
  4. SPI DMA channel conflict possible between FPGA (SPI3) and SD card
  5. GPIO toggle hard floor: 5 ms (100 Hz max) — document in API

=== CURRENT TASK ===
[Describe exactly what you are working on right now.]
[Example: "I am porting powerMonitor.c to Zephyr. Here is the current HAL version.
 Replace the I2C calls and mutex with Zephyr equivalents. Keep the public API
 identical. Add complete Doxygen comments to all public functions."]
```

```
CONSTRAINTS FOR THIS TASK:
  - Target RTOS: [FreeRTOS / Zephyr]
  - Target board: [inklab_v1 / inklab_v2 / nucleo_h562zi]
  - Files being modified: [list files]
  - Do not change: [list anything that must stay the same]
```

6.1 Prompt Usage Tips

- Always paste the full context block above, not just the CURRENT TASK section. The model needs the full picture to avoid generic advice.
- When porting a module, paste the ENTIRE existing .h and .c file into the prompt after the context block. Do not summarize it.
- If the model suggests a function that you cannot find in the Zephyr docs, ask: "Show me the Kconfig symbol that enables this API and the DTS property it depends on."
- After any AI-generated code, add: "Now add complete Doxygen comments (@brief, @param, @retval, @note) to every public function in this file."
- When debugging: paste the exact west build error output. Do not paraphrase it.

7. Quick Reference: HAL → Zephyr API Cheat Sheet

STM32 HAL / FreeRTOS	Zephyr Replacement
HAL_GPIO_WritePin(port, pin, val)	gpio_pin_set_dt(&spec, val)
HAL_GPIO_ReadPin(port, pin)	gpio_pin_get_dt(&spec)
HAL_I2C_Mem_Read(...)	i2c_burst_read_dt(&dev, reg, buf, len)
HAL_I2C_Mem_Write(...)	i2c_burst_write_dt(&dev, reg, buf, len)
HAL_SPI_Transmit_DMA(...)	spi_transceive_dt(&dev, &tx_set, &rx_set)
HAL_TIM_PWM_Start(...)	pwm_set_dt(&spec, period, pulse)
HAL_ADC_Start / HAL_ADC_PollForConversion	adc_sequence_init_dt + adc_read
osDelay(ms)	k_sleep(K_MSEC(ms))
osThreadNew(fn, arg, &attr)	K_THREAD_DEFINE(id, stack, fn, ...)
osMessageQueueNew(n, sz, attr)	K_MSGQ_DEFINE(q, sz, n, align)
osMessageQueuePut(q, &msg, 0, timeout)	k_msgq_put(&q, &msg, K_MSEC(timeout))
osMessageQueueGet(q, &msg, 0, timeout)	k_msgq_get(&q, &msg, K_MSEC(timeout))
osMutexNew(attr)	K_MUTEX_DEFINE(m) or k_mutex_init(&m)
osMutexAcquire(m, timeout)	k_mutex_lock(&m, K_MSEC(timeout))
osMutexRelease(m)	k_mutex_unlock(&m)

STM32 HAL / FreeRTOS	Zephyr Replacement
CDC_Transmit_FS(buf, len)	uart_fifo_fill(cdc_dev, buf, len)
f_mount / f_open / f_read (FATFS)	fs_mount + fs_open + fs_read (same ELM FatFs)
__disable_irq() / __enable_irq()	irq_lock() / irq_unlock()
vTaskGetRunTimeStats()	CONFIG_THREAD_ANALYZER=y + thread_analyzer_print()

InkLab Zephyr Porting Guide — End of Document